

第1章 绪论

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 复习全书的地基数学:指数、对数、级数、模运算及其算法用途
- 理解"程序 = 数据结构 + 算法",建立正确性与效率维度,预告复杂度记号
- 掌握**递归的四法则**,能写出阶乘等经典递归
- 完成 C 到 Python 的第一波迁移:print/input、list、def、class
- 亲手实现斐波那契的朴素递归与记忆化两版,体会 $O(2^n)$ 与 $O(n)$ 之别
- 掌握 RecursionError 的成因与 sys.setrecursionlimit 的用法和危险
- 学完能独立实现汉诺塔与全排列(章末实验),验证 $2^n - 1$ 与 $n!$

💡 从 C 到 Python:本章主题如何迁移

全书一句话直觉:数据结构决定"数据怎么放",算法决定"问题怎么做",两者合起来决定"程序跑多快"。第1章是唯一的地基章,不打地基,后面 11 章都要还债。

语言迁移上,你的 C 技能大部分直接平移:循环、分支、函数——对应,差异在"形态":块用缩进、变量不声明类型、数组换成了自动扩容的 list。本章只补四样新工具:print/input、list(动态数组 + 切片)、def、class;先学会它们,第3章起才能专心学数据结构。

Python 有两个 C 没有的独特优势:整数任意精度(永不溢出)与**解释器自动管理内存**(不用 malloc/free)。代价是同样代码通常比 C++ 慢 30-100 倍,但增长阶是语言无关的(第2章):Python 的 $O(n)$ 照样碾压 C 的 $O(n^2)$ 。

算法直觉上,本章最重要的思想是**递归**:把大问题缩小成"同样结构的小问题"。同一个斐波那契,笨写是指数级,加一行"记忆化"就变成线性——全书第一次"复杂度决定生死"的现场演示;Python 的 functools.lru_cache 让记忆化一行实现。

📦 前置知识自查

数学前置:高中数学:幂运算与对数(§1.1 系统复习);求和记号 Σ 会读即可(§1.1 给出全部公式)。

C 语言前置:函数定义与调用、for/while 循环、数组与下标、printf/scanf、struct 与指针(会更好)。

依赖的前面章节:无,本章是全书第一章。

🚀 本章学习路线

1. **第一阶段(1 小时):**读 §1.1-§1.2,在 REPL 里验证指数/对数/级数/取模的例子,背下里程碑数值
2. **第二阶段(1.5 小时):**读 §1.3-§1.4,把 §1.4 的完整小程序存为 hello.py 并运行、改造
3. **第三阶段(1.5 小时):**读 §1.5-§1.6,亲手运行朴素与记忆化斐波那契,观察时间如何爆炸
4. **第四阶段(实验,1 小时):**完成章末实验汉诺塔与全排列,验证 $2^n - 1$ 与 $n!$
5. **验收标准:**能默写递归四法则,独立写出朴素与记忆化斐波那契,解释 RecursionError 与 setrecursionlimit 的危险

本章知识导图

- §1.1 数学复习:指数、对数与换底、级数三公式、负数取模(C/Python 差异)
- §1.2 算法概念:程序 = 数据结构 + 算法、正确性、效率、复杂度预告
- §1.3 递归简介:递归四法则、调用栈回顾、阶乘示例
- §1.4 从 C 到 Python 速成:print/input、list、def、class、完整小程序
- §1.5 完整示例:斐波那契——朴素 vs 记忆化、手写 vs lru_cache
- §1.6 递归的工程注意:RecursionError、setrecursionlimit 的用法与危险
- §1.7 全书路线图:12 章一句话导航

§1.1 数学复习

1.1.1 指数规则

指数是全书最频繁出现的运算,下列规则必须刻进直觉,后面推导复杂度时随时在用:

- $a^0 = 1 (a \neq 0); a^1 = a; a^{-n} = 1/a^n$
- **同底数相乘,指数相加:** $a^m \cdot a^n = a^{m+n}$
- **幂的幂,指数相乘:** $(a^m)^n = a^{m \cdot n}$
- **同底数相除,指数相减:** $a^m / a^n = a^{m-n}$

背几个里程碑数值: $2^{10} = 1024; 2^{20} \approx 10^6; 2^{30} \approx 10^9; 2^{40} \approx 10^{12}$ 。反过来记: $10^6 \approx 2^{20}$,意味着对 100 万规模做"每次减半"的查找最多约 20 步——这是第2章对数直觉的起点。可在 REPL 用 `2 ** 30` 验证。

记号说明(全书约定,首次出现)

本书数学记号统一如下:幂用上标,如 n^2 、 2^n ;对数的底数用下标,如 $\log_2 n$;递推式行内用等宽代码,如 $T(n) = T(n-1) + O(1)$;求和用希腊字母 Σ ,如 $\Sigma_{i=1}^n i$;模运算在正文写 $a \bmod b$,在代码里一律写语言语法 `a % b`。代码与数学记号严格分离:代码里只用 `%`、`**`。

1.1.2 对数

对数的定义: $\log_b X = a$ 当且仅当 $b^a = X (b > 0, b \neq 1, X > 0)$ 。一句话直觉:对数回答"几次方"——"2 的几次方等于 8",答 $\log_2 8 = 3$ 。对数与指数互为逆运算,是"减半""倍增"类算法(二分查找、快速幂、欧几里得算法)的天然度量。

换底公式(全书最重要的对数公式): $\log_b X = \log_a X / \log_a b$ 。推导:设 $\log_b X = c$,则 $b^c = X$,两边取对数得 $c \cdot \log_a b = \log_a X$ 。工程含义:换底只差常数倍,而常数倍会被大 O 记号吞掉(第2章),所以**底数在复杂度分析里不重要**。

记法约定:计算机领域 $\log$ 默认以 2 为底; $\ln$ 是以 $e \approx 2.718$ 为底; $\lg$ 有时指以 10 为底。三个里程碑: $\log_2 1024 = 10; \log_2 10^6 \approx 20; \log_2 10^9 \approx 30$ 。对数"几乎不增长":规模翻 1000 倍, $\log_2 n$ 只从 10 长到 20——对数级算法都值得信赖。

1.1.3 级数

三个级数公式是第2章复杂度计算的原料:

- **算术级数:** $\sum_{i=1}^n i = n(n+1)/2 \approx n^2/2$ ——双重循环的总迭代次数,量级 $O(n^2)$ 。例: $1+2+\dots+100 = 5050$
- **几何级数:** $\sum_{i=0}^n 2^i = 2^{n+1} - 1$ ——倍增翻倍的总量,量级 $O(2^n)$ 。例: $1+2+4+8+16+32 = 63$;推广: $\sum_{i=0}^n r^i = (r^{n+1} - 1)/(r-1)$
- **调和级数:** $\sum_{i=1}^n 1/i \approx \ln n$ ——量级 $O(\log n)$,出现在快排平均分析中

1.1.4 模运算

模运算: $a \bmod b$ 表示 a 除以 b 的余数,数学上写成 $a = q \cdot b + r$ (q 为商, r 为余数);"余数"取决于"商怎么取整",这正是各语言分歧的根源(★见下)。两条最重要的性质(散列、循环数组下标、快速幂的全部基础):

- $(a+b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$
- $(a \cdot b) \bmod m = ((a \bmod m) \cdot (b \bmod m)) \bmod m$

★ 重点:Python 与 C 的负数取模不一样

取模的关键分歧在**负数**:C 除法"向 0 截断",余数符号与被除数相同;Python 除法"向下取整",余数符号与除数相同。同一表达式,两种语言答案不同:

C 的写法(商向 0 截断)	Python 的写法(商向负无穷取整)
<code>-7 % 3</code> 结果是 -1 (<code>-7/3</code> 截断为 <code>-2</code> ,余数 <code>-7 - (-2)·3 = -1</code>)	<code>-7 % 3</code> 结果是 2 (<code>-7/3</code> 向下取整为 <code>-3</code> ,余数 <code>-7 - (-3)·3 = 2</code>)
<code>7 % -3</code> 结果是 1 (<code>-2</code> 为商,1 为余数)	<code>7 % -3</code> 结果是 -2 (<code>-3</code> 为商, <code>-2</code> 为余数)
<code>-7 / 3</code> 结果是 -2 (截断)	<code>-7 // 3</code> 结果是 -3 (向下取整)

两者在"同号"时一致,只在异号时分歧。判奇偶、判整除不受影响——受影响的是"余数的正负与大小"。本书代码都在"非负输入"语境中使用 `%`,但从 C 移植时会悄悄改变结果,务必知道这个坑。

⚠ 误区 1:把 C 的取模语义带进 Python

Python 的 `-7 % 3` 是 **2** 而不是 `-1`:商向负无穷取整,余数恒与除数同号(除数非负时恒非负)。从 C 移植含负数除法的代码,结果可能悄悄改变: `a % b == 0` 的整除判断不变,但 `a / b` 的符号行为完全不同(C 截断,Python `//` 向下取整),需要 C 语义用 `math.trunc(a / b)`。另一个专属坑: `5 / 2` 得到浮点 `2.5`——真除法与整除(`//`)是两回事。

⚠ 误区 2:int 溢出?Python 不存在

C 的 `int` 是 32 位, `INT_MAX + 1` 回绕成负数或是未定义行为;斐波那契 `F(47)` 就超界(§1.5)。Python 的整数是**任意精度**的: `21000`、`1000!` 这种 C 中早已溢出的数,Python 精确到每一位。代价:大整数运算比固定宽度慢,每个整数都是堆上对象,同样 $O(n)$ 循环通常比 C++ 慢 30–100 倍。

示例 1-1 Python 负数取模实测

```
# Python 与 C 不同:取模结果是"向下取整"除法的余数
# 公式:余数 = 被除数 - 商 × 除数,商向负无穷取整
print(-7 % 3)      # 2:-7/3 = -2.33,商向下取整为 -3,余数 -7 - (-3)*3 = 2
print(7 % -3)       # -2:商向下取整为 -3,余数 7 - (-3)*(-3) = -2
print(-7 // 3)      # -3:整除同样向下取整(与 C 的截断不同!)
print(-7 / 3)       # -2.3333333333333335:真除法,结果总是浮点数
print(7 % 3, -7 % -3) # 1 -1:同号时与 C 的行为一致
print((-7) % 3)     # 2:加括号防混淆,与 print(-7 % 3) 结果相同
```

§1.2 算法概念

1.2.1 程序 = 数据结构 + 算法

瑞士计算机科学家 Niklaus Wirth 的名言: **程序 = 数据结构 + 算法**。**数据结构**回答"数据如何组织",**算法**回答"问题如何求解",两者互相成就:同一份数据,组织方式不同,解法难度天差地别。经典例子——电话簿:按姓名排序可以二分查找(第2章)瞬间定位;打乱顺序只能逐个翻。

1.2.2 正确性与效率

一个解法要配得上"算法"二字,必须同时满足两条:**正确性**——对一切合法输入给出正确输出;**效率**——时间与空间在可接受范围内。正确但慢到跑不完等于不能用,快但错比慢更危险。

例:求 2^n ,连乘 n 次——正确,但 $n = 10^6$ 时要一百万次乘法;第2章的快速幂约 20 次搞定。本书的尺度:先保证正确,再追求高效;复杂度分析(第2章)给"效率"机器无关的精确度量,本章只需建立直觉:数据规模变大时,运行时间如何增长。

1.2.3 复杂度记号预告

全书用大 O 记号描述效率。先混个脸熟,第2章给严格定义:

- $O(1)$ 常数:步数与规模无关——按下标取列表元素
- $O(\log n)$ 对数:规模翻倍只多一步
- $O(n)$ 线性:遍历一趟
- $O(n \log n)$ 线性对数:归并排序
- $O(n^2)$ 平方:双重循环——插入排序
- $O(2^n)$ 指数:穷举、朴素斐波那契,稍大就不可用

增长阶排序(必须背诵):常数 < 对数 < 线性 < 线性对数 < 平方 < 立方 < 指数。本章预用两个结论(方法见第2章):**递推式 $T(n) = T(n-1) + O(1)$ 的解是 $O(n)$** (阶乘),**朴素斐波那契的递推式 $T(n) = T(n-1) + T(n-2) + O(1)$ 的解是 $O(2^n)$** 。

§1.3 递归简介

1.3.1 递归四法则

Weiss 教材把递归写成四条法则,每条都配一个"违反后果":

1. **基准情形(base case)**:存在不需要递归、直接得出答案的情形。没有基准情形 = 无限递归,程序永不停止。
2. **不断推进**:每次递归调用都必须让问题朝基准情形靠近。不推进 = 死循环;推进太快跳过基准情形 = 边界错误。

3. **设计法则**:假设所有递归调用都能正确工作,只设计"当前层如何利用子问题的答案"。不要手工跟踪每一层——人脑栈只有几层深,程序栈可以有上千层。
4. **合成效益法则**:不要在递归调用的不同分支中重复求解同一个问题实例。同一子问题被算很多遍,复杂度会爆炸成指数级——§1.5 的朴素斐波那契正是反面教材。

1.3.2 C 递归与调用栈回顾

你学 C 时写过递归,现在补上底层机制。程序运行时,系统维护一块**调用栈**(call stack):每次函数调用,系统把"返回地址 + 参数 + 局部变量"打包成一个**栈帧**(stack frame)压入调用栈;函数返回时弹出,继续执行调用点的下一条语句。递归调用也是普通调用,只是被调用的函数与自己同名。

由此得出两个推论:① **递归深度 = 同时存活(尚未返回)的栈帧数**;② 递归占用额外内存(每层一个栈帧)。C 的调用栈约 8MB,深度过大时进程静默崩溃;Python 用"递归深度上限 + RecursionError"显式保护——这是两版刻意不同的工程注意点(§1.6 详解)。

C 与 Python 的递归语法几乎一致:定义、调用、参数传递同构;差异在函数体靠缩进分层,且默认深度上限只有 1000(§1.6)。

1.3.3 阶乘示例

阶乘 $n! = n \times (n-1) \times \dots \times 1$,基准情形 $0! = 1$ 。递归定义: $n! = n \cdot (n-1)!$ ——线性递归的标准形态,递推式 $T(n) = T(n-1) + O(1)$,解为 $O(n)$ 。

示例 1-2 阶乘:递归版与迭代版

```
# 递归版:n! = n * (n-1)!, n <= 1 是基准情形
# 与 C 的写法几乎一致:递归是语言无关的思想
def fact_rec(n):
    if n <= 1:
        return 1
    return n * fact_rec(n - 1)

# 迭代版:循环乘过去,不占调用栈
def fact_iter(n):
    r = 1
    for i in range(2, n + 1):
        r *= i
    return r

# 验证:两种写法一致,且 100! 不溢出(任意精度)
print(fact_rec(10), fact_iter(10))
print(len(str(fact_iter(100))))
```

基准情形: $0! = 1! = 1$
不断推进: n 越来越小
3628800 3628800
158:100! 有 158 位数字

§1.4 从 C 到 Python 速成

1.4.1 输入输出:printf/scanf → print/input

Python 的 `print()` 替代 `printf`, `input()` 替代 `scanf`: `print(x, y)` 自动以空格分隔并换行; `input()` 读取**一整行字符串**并去掉末尾换行。最重要的差异: `input()` 返回字符串,读数字必须手动转换——`int(input())` 读整数、`float(input())` 读浮点数,忘记转换是入门第一坑(C 的 `%d` 自动解析,Python 必须显式)。

C 的写法	Python 的写法
<code>printf("n = %d\n", n);</code>	<code>print(f"n = {n}")</code> 或 <code>print("n =", n)</code>
<code>scanf("%d", &n);</code>	<code>n = int(input())</code>
<code>scanf("%s %d", name, &score);</code>	<code>name, score = input().split()</code>
格式符写错 = 静默读错/崩溃	类型不匹配直接抛异常,显式暴露

f 字符串(`f"平均分: {avg}"`)是格式化输出的标准姿势,省略了 C 的 `%d` 位置参数。

1.4.2 动态数组与字符串:malloc 数组/char* → list/str

C 里动态数组要自己管"长度、容量、扩容、释放"四件事;Python 的 `list` 全部内置:自动扩容、垃圾回收、自带 `len()`。字符串 `str` 是不可变字符序列,操作对应 C 的 `<string.h>` 函数。学习阶段要理解 list 的底层(第3章会手写一遍倍增扩容),工程阶段直接用。

C 的写法	Python 的写法
<code>int *a = malloc(n * sizeof(int));</code>	<code>a = [0] * n</code> 或 <code>a = []</code>
<code>a[len++] = x;</code> (自己管长度)	<code>a.append(x)</code> (自动扩容)
<code>strcpy(buf, s); strcat(buf, t);</code>	<code>s = "hello"; s += " world"</code>
<code>free(a);</code> (忘记即泄漏)	垃圾回收器自动释放
<code>a[i]</code> 越界 = 未定义行为(可能崩溃)	<code>a[i]</code> 越界抛 <code>IndexError</code> ,显式暴露

示例 1-3 list 与字符串速览

```
s = "hello"           # 字符串:不可变字符序列
s += " world"         # 拼接(产生新字符串)
print(len(s), s[0])   # 11 h
v = [3, 1, 4]         # 列表:自动扩容的动态数组
v.append(1)           # 尾部插入
v[0] = 7              # 按下标访问 0(1)
print(len(v), v[0])   # 4 7
for x in v:           # 遍历
    print(x, end=" ")
print(v[1:3])         # 切片:取下标 1..2 的视图
print(sorted(v))       # 排序返回新列表(第7章)
```

1.4.3 函数:C 的函数 → def

Python 函数用 `def 函数名(参数):` 定义,函数体靠缩进分层(C 靠花括号)。与 C 的三个差异:① 参数与返回值都不写类型;② 可有默认参数 `def f(a, b=1)`;③ 返回多个值用元组 `return a, b`,调用处 `x, y = f()` 解包。递归写法与 C 同构——定义里调用自己即可。函数是一等对象,可当参数传递(第10章,§1.5 实验会用到)。

1.4.4 类入门:struct+函数 → class

C 里"数据 + 操作"分离:struct 存数据,全局函数操作它,每次调用都要传指针。Python 的 `class` 把数据与操作封装成整体,方法(成员函数)的第一个参数固定是 `self` (指对象自己)。这是全书所有数据结构(栈、树、堆.....)的呈现形式,第3章还会从 ADT 角度深化。

示例 1-4 类入门:计数器

```
# C 的做法:struct + 全局函数 counter_inc(&c)
# Python 的做法:class 把数据与操作打包,self 指对象自己
class Counter:
    def __init__(self):          # 构造函数:创建对象时自动调用
        self.cnt = 0           # 成员变量

    def inc(self):              # 方法:操作自己的数据
        self.cnt += 1

    def get(self):
        return self.cnt

c = Counter()                  # C 里要 counter_init(&c),这里自动执行
c.inc()
c.inc()
print(c.get())                 # 2
```

1.4.5 运行与完整小程序

Python 不需要编译:保存为 `hello.py` 后运行 `python hello.py`,或输入 `python` 进入 REPL 逐行试验。程序入口习惯用 `if __name__ == "__main__":` 包裹主流程,既能直接运行、也能被 `import`(第10章用到)。

示例 1-5 完整小程序:学生成绩统计(综合本节全部知识点)

```
# hello.py — 第一个完整 Python 程序:学生成绩统计
# 输入:姓名与分数;输出:平均分与最高分
class Student:
    def __init__(self, name, score): # 构造函数
        self.name = name           # 成员变量
        self.score = score

def main():
    n = int(input("学生人数: "))    # input 返回字符串,必须转 int
    stu = []                        # 列表,存 Student 对象
    for _ in range(n):
        name, score = input().split() # "张三 90" -> ["张三", "90"]
        stu.append(Student(name, int(score)))
    total = 0
    best = 0
    for s in stu:                   # 遍历
        total += s.score
        if s.score > best:
            best = s.score
    print(f"平均分: {total // n},最高分: {best}")

if __name__ == "__main__":
    main()

# 运行命令:python hello.py
# 运行示例:
# 学生人数: 3
# 张三 90 / 李四 85 / 王五 95
# 平均分: 90,最高分: 95
```

§1.5 完整示例:斐波那契

1.5.1 朴素递归:指数级

斐波那契数列: $F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2)$ 。照定义直接写递归——这是教科书般的"正确但低效":

示例 1-6 朴素递归斐波那契, $O(2^n)$

```
# F(0)=0, F(1)=1, F(n) = F(n-1) + F(n-2)
# 朴素递归:同一个 F(k) 被重复计算无数遍(违反合成效益法则)
def fib_naive(n):
    if n <= 1:
        return n                # 基准情形
    return fib_naive(n - 1) + fib_naive(n - 2) # 不断推进,但一裂为二

# 调用次数 = 2*F(n+1) - 1(F(40) 约 3.3 亿次)
# 递推式为  $O(2^n)$ ; n 每加 10, 调用数约乘 120
```

⚠ 误区 3:"递归只是慢一点"——是数量级爆炸

同一输入,朴素版 $F(40)$ 约 3.3 亿次调用(本机实测 $F(34) \approx 0.43$ 秒,推算 $F(40) \approx 8$ 秒),记忆化版(下节)约 41 次调用,相差约八百万倍。指数级增长意味着:输入每加一点,时间乘一个常数—— n 从 40 到 50,调用数再乘约 120($\varphi^{10} \approx 123$),任何现实规模都不成立。另外,朴素版在 n 接近 1000 时还会触发 `RecursionError`(§1.6)——"指数级 + 深递归"是双重灾难。

1.5.2 记忆化:线性

观察调用树: $F(5)$ 要算 $F(4)$ 、 $F(3)$, $F(4)$ 又要算 $F(3)$ 、 $F(2)$ —— $F(3)$ 被重复计算了两次, $F(2)$ 更多次。解决办法就是**记忆化**(memoization):第一次算出 $F(k)$ 就存进表格,以后再要直接查表。每个 $F(k)$ 只算一次,共 $n+1$ 个不同子问题,每个 $O(1)$,总计 $O(n)$ 。

示例 1-7 手写记忆化 vs 标准库 `lru_cache`

```

from functools import lru_cache

# 手写记忆化:用字典记住已经算过的结果
memo = {}
def fib_hand(n):
    if n <= 1:
        return n
    if n not in memo:
        # 没算过才算,算过直接取
        memo[n] = fib_hand(n - 1) + fib_hand(n - 2)
    return memo[n]

# 标准库写法:lru_cache 一行实现同样缓存
@lru_cache(maxsize=None)
def fib_cache(n):
    if n <= 1:
        return n
    return fib_cache(n - 1) + fib_cache(n - 2)

# 两种写法结果一致,F(100) 瞬间算完(C 的 long long 早已溢出)
print(fib_hand(100))          # 354224848179261915075
print(fib_cache(100))        # 354224848179261915075

```

1.5.3 手写 vs 标准库

上面并排展示了本章"手写 + 标准库"的范例:手写版用字典显式管理缓存,让你看清机制; `lru_cache` 一行实现——内部正是手写版的机制(带"最近最少使用"淘汰策略,C/C++ 没有直接对应物)。工程中:单个函数需要缓存直接用 `@lru_cache`;需要控制缓存生命周期(如按测试用例清空)时手写字典。两者都要会:看懂手写版才能排查 `lru_cache` 的问题。

1.5.4 迭代版:空间也省下来

斐波那契还能更省:只保留前两个值滚动前进,时间 $O(n)$ 、空间 $O(1)$: `a, b = 0, 1`, 循环 `a, b = b, a + b` (元组同时赋值,C 里要写三行临时变量)。对比三版:朴素 $O(2^n)$ 时间、记忆化 $O(n)$ 时间 $O(n)$ 空间、迭代 $O(n)$ 时间 $O(1)$ 空间。记忆化解决"重复计算",迭代解决"重复计算 + 栈空间"——能迭代时优先迭代。用迭代算 $F(10^6)$ 也只需几十毫秒。

§1.6 递归的工程注意

1.6.1 RecursionError 与递归深度上限

与 C/C++ 的"静默段错误"不同,Python 用显式保护代替崩溃:递归深度默认上限 `sys.getrecursionlimit() = 1000`,超过即抛 `RecursionError`——这是**好事**:问题在错误堆栈里暴露得一清二楚,而不是进程无声消失。

示例 1-8 RecursionError 现场

```
# 危险代码!运行会抛 RecursionError
def count_down(n):
    print(n)
    count_down(n - 1)      # 永不停止,直到超过递归深度上限

# count_down(10**6)
# 打印到 996 附近时抛出异常:
# RecursionError: maximum recursion depth exceeded
# 到达上限才报错,不是立刻崩溃
```

⚠ 误区 4:递归深度 = 递归"层数",不是 n 的位数

常见误解:递归 $n = 10^6$ 只是"数字大一点"。事实:线性递归的深度就是 n —— $F(10^6)$ 需要 10^6 层栈帧,远超 1000 上限,必然 RecursionError。凡是深度可能超过约 1000 的递归,在 Python 里都写不了,必须改迭代或分治(log 深度永远安全,第2章)。

1.6.2 sys.setrecursionlimit:用法与危险

上限可以调大: `sys.setrecursionlimit(20000)` 提到 2 万层。合法场景:"深度合理但超过默认值"的递归——例如对 10^4 规模的链表递归遍历(第3、4章的树,深度 = 树高)。

示例 1-9 setrecursionlimit 的用法与边界

```
import sys
print(sys.getrecursionlimit())      # 默认 1000

def sum_rec(n):
    if n == 0:
        return 0
    return n + sum_rec(n - 1)

sys.setrecursionlimit(20000)        # 调大到 2 万层
print(sum_rec(10000))               # 50005000:深度 1 万,轻松完成
# 危险:若调到  $10^6$  并真的递归  $10^6$  层——
# ① 解释器自身的 C 栈先耗尽,进程直接段错误崩溃,
#   连 try/except 都拦不住;
# ② 深度  $10^6$  可能吃掉数百 MB 内存;
# ③ RecursionError 这道"安全网"失效
```

🚀 工程建议:setrecursionlimit 的三条纪律

① 只小幅调高(默认值 10–20 倍以内),调大不调小;② 调大后仍用迭代优先的算法设计,把递归当作"深度可控的例外";③ 宁可写显式栈(第3章)也不赌大上限——显式栈深度只受内存限制。记住: `setrecursionlimit` 是"最后一根稻草",不是常规工具。

1.6.3 如何避免深层递归

避免深层递归的三招:① **循环改写**——尾递归(递归调用在末尾)最容易改写,空间从 $O(n)$ 变 $O(1)$,§1.5.4 的迭代斐波那契就是例子;② **显式栈**——自己维护的列表模拟系统调用栈(第3章),不受深度限制;③ **控制深度**——"减半"式递归(分治,第2章)深度只有 $\log_2 n$, 10^6 规模也不到 20 层,永远安全。

§1.7 全书路线图

全书 12 章,一句话导航:

- **第2章 算法分析:**大 O 记号、运行时间计算
- **第3章 表、栈和队列:**三种最基础 ADT,两种实现
- **第4章 树:**遍历、表达式树、BST、AVL
- **第5章 散列:**散列函数、分离链接与开放定址
- **第6章 优先队列(堆):**二叉堆、堆排序
- **第7章 排序:**插入/希尔/堆/归并/快排/桶排
- **第8章 不相交集:**按秩合并与路径压缩的并查集
- **第9章 图论算法:**DFS/BFS、拓扑排序、最短路径、最小生成树
- **第10章 算法设计技巧:**贪心、分治、动态规划、回溯、随机化
- **第11章 摊还分析:**聚集、会计、势能三种方法
- **第12章 高级数据结构:**伸展树、B 树、跳表

依赖关系:第2章是第3章起所有复杂度结论的依赖,务必学透;第3–9章是"结构线",第10章是"算法线",两线在第10章汇合;第11章为摊还补上严格证明。每章结束时回看这张图。

本章复杂度速查表

操作 / 算法	数据结构 / 算法	平均	最坏
求 a^n	朴素连乘(手写循环)	$O(n)$	$O(n)$
求 a^n	快速幂(第2章)	$O(\log n)$	$O(\log n)$
求 $\log_b X$	换底公式 + 库函数 $\log$	$O(1)$	$O(1)$
级数求和	算术级数(公式直接算)	$O(1)$	$O(1)$
级数求和	几何级数(公式直接算)	$O(1)$	$O(1)$
级数求和	调和级数(逐项累加)	$O(n)$	$O(n)$
整数取模	C(商向 0 截断)	$O(1)$	$O(1)$
整数取模	Python(商向下取整)	$O(1)$	$O(1)$
阶乘 $n!$	递归(线性递归)	$O(n)$	$O(n)$
阶乘 $n!$	迭代循环	$O(n)$	$O(n)$
斐波那契 $F(n)$	朴素递归(违反合成效益法则)	$O(2^n)$	$O(2^n)$
斐波那契 $F(n)$	记忆化递归	$O(n)$	$O(n)$
斐波那契 $F(n)$	迭代滚动(两个变量)	$O(n)$	$O(n)$
递归调用	递推式 $T(n) = T(n-1) + O(1)$	$O(n)$	$O(n)$
递归调用	递推式 $T(n) = T(n-1) + T(n-2) + O(1)$	$O(2^n)$	$O(2^n)$
递归深度	Python 默认上限(RecursionError)	1000 层	setrecursionlimit 可调大,但过大有崩溃风险
调用栈空间	C/C++ 递归深度 n (默认栈约 8MB)	$O(n)$ 栈空间	深度过大即段错误

最小必会清单

- 能默写指数四条规则与里程碑数值
- 能说出对数定义、换底公式与 $\log/\log_2/\ln$ 约定,并解释"底数不重要"
- 能默写算术、几何、调和三个级数公式并说出各自量级
- 能说出 Python 与 C 负数取模的区别(Python 向下取整, $-7\%3=2$; C 截断, $-7\%3=-1$)与"整除 vs 真除法"
- 能说出 Python 整数任意精度与 C 的 int 溢出的对比
- 能说出"程序 = 数据结构 + 算法",举例说明数据结构如何影响算法难度
- 能默写递归四法则,并指出朴素斐波那契违反的是合成效益法则
- 能独立写出阶乘的递归版与迭代版,说出 $T(n) = T(n-1) + O(1)$ 的解
- 能独立写出朴素与记忆化斐波那契(手写字典版与 @lru_cache 版),说出各自复杂度
- 能解释 RecursionError 与 setrecursionlimit 的用法和危险
- 能用 python hello.py 运行一个含 input/print、list、def、class 的完整程序
- 能说出本书 12 章的大致内容与依赖关系

自测题

Q 1. 计算: 2^{10} 是多少? $\log_2 1024$ 是多少? $\sum_{i=1}^{100} i$ 是多少?

✓ 参考答案

$2^{10} = 1024$; $\log_2 1024 = 10$; $\sum_{i=1}^{100} i = 100 \times 101 / 2 = 5050$ 。对应指数、对数、算术级数三个公式。

Q 2. Python 中 $-7 \% 3$ 与 $7 \% -3$ 分别等于多少?与 C 有何不同?

✓ 参考答案

$-7 \% 3 = 2$, $7 \% -3 = -2$ 。Python 除法向下取整: $-7/3$ 取商 -3 , 余数 $-7 - (-3) \times 3 = 2$; $7/-3$ 取商 -3 , 余数 $7 - (-3) \times (-3) = -2$; C 向 0 截断, 分别为 -1 与 1 。同号一致, 异号不同; 判奇偶、判整除不受影响。

Q 3. 递归四法则是什么?朴素递归斐波那契违反了哪一条?

✓ 参考答案

① 基准情形: 有不需递归的直接答案; ② 不断推进: 每次调用都朝基准情形靠近; ③ 设计法则: 假设所有递归调用都正确, 只组装当前层; ④ 合成效益法则: 不在调用间重复求解同一问题实例。朴素斐波那契违反第④条, 复杂度爆炸到 $O(2^n)$ 。

Q 4. 递推式 $T(n) = T(n-1) + O(1)$ 的解是什么?给出直觉推导。

✓ 参考答案

解为 $O(n)$ 。展开: $T(n) = T(n-1) + c = T(n-2) + 2c = \dots = T(0) + n \cdot c$, 共 n 步、每步常数代价。阶乘与递归求和都是这个递推式。

Q 5. 朴素递归斐波那契计算 $F(40)$ 大约需要多少次递归调用?记忆化后呢?为什么 Python 里还可能先遇到 `RecursionError`?

✓ 参考答案

朴素版调用次数 $= 2 \times F(41) - 1 \approx 3.3$ 亿次(本机实测约 8 秒); 记忆化后约 41 次调用, 瞬间完成。但 $n = 40$ 时递归深度只有 40, 不会触发 1000 层上限, 只有 n 接近 1000 才会先撞 `RecursionError`。更现实的风险是: 指数级调用本身就慢到不可用。

Q 6. Python 默认递归深度上限是多少?把 `sys.setrecursionlimit` 调得过大有什么危险?

✓ 参考答案

默认 1000 层(`sys.getrecursionlimit()` 查看)。危险: ① 调得过大(如 10^6)并真递归到那个深度时, 解释器自身的 C 栈先耗尽, 进程段错误崩溃, 连 `try/except` 都拦不住; ② 每层帧内存开销可观; ③ `RecursionError` 这道安全网失效。正确姿势: 能改迭代就改迭代, `setrecursionlimit` 只小幅调高。

章末实验题:递归综合实验:汉诺塔与全排列

实验 1:汉诺塔与全排列(递归综合)

实验目标:用递归实现汉诺塔与全排列,验证 $2^n - 1$ 与 $n!$ 两个计数结论,并用真实计时感受朴素斐波那契的指数级增长。

实验要求:

- 任务 1:汉诺塔——递归打印移动步骤、统计总步数,验证 $n = 1..10$ 步数恒为 $2^n - 1$ (知识点:§1.3 递归四法则、基准情形)
- 任务 2:递归生成全排列——交换 + 回溯,验证 $n = 3、4$ 的排列总数为 $3! = 6、4! = 24$ (知识点:§1.3 树形递归、回溯思想)
- 任务 3:对比 n 增大时朴素递归斐波那契的时间增长——记录 $n = 28/31/34$ 的耗时,验证 n 加 3 时耗时约乘 4~5(调用数比值恰为 $\varphi^3 \approx 4.2$;知识点:§1.5 指数级复杂度、§1.2 效率直觉)

实验环境:Python 3.8+。运行: `python lab1.py`。

第一步 写 `hanoi`:基准情形 $n == 1$ (只剩一个盘子,直接搬);否则三步——上面 $n-1$ 个搬到中转柱、搬最大盘、再搬 $n-1$ 个。注意三次调用中参数轮换的顺序;用全局变量统计步数(先声明 `global`)

第二步 开关只对 $n = 3$ 打开, $n = 1..10$ 只输出"实际 vs 理论 $2^n - 1$ "对照行

第三步 `permute`:固定前 k 位;for 循环把 `a[k..]` 每个元素换到第 k 位;递归排列剩余部分;换回来还原现场(回溯三件套);基准情形 $k == n$ 时用 `a[:]` 拷贝当前排列

第四步 时用 `time.perf_counter()` (§2.1 系统讲),对 $n = 28/31/34$ 各测一次,算"每加 3 乘几"的比值

✅ 参考答案(完整可运行程序,分两段)

lab1.py(第一段:任务 1 汉诺塔与任务 2 全排列)

```
# lab1.py — 递归综合实验(第一段:任务1与任务2)
import time

move_cnt = 0                                # 汉诺塔移动总步数

# 任务 1:汉诺塔。print_move=True 时打印每步
def hanoi(n, src, via, dst, print_move):
    global move_cnt
    if n == 1:                               # 基准情形:只剩一个盘子,直接搬
        if print_move:
            print(f" {src} -> {dst}")
        move_cnt += 1
        return
    hanoi(n - 1, src, dst, via, print_move)  # 1. 上面 n-1 个搬到中转柱
    if print_move:
        print(f" {src} -> {dst}")
    move_cnt += 1                             # 2. 最大盘子搬到目标柱
    hanoi(n - 1, via, src, dst, print_move)  # 3. n-1 个从中转柱搬到目标

# 任务 2:全排列(回溯)。固定前 k 位,把 a[k..] 逐个换到 a[k]
def permute(a, k, result):
    if k == len(a):                           # 基准情形:最后一位也固定了
        result.append(a[:])                  # 拷贝当前排列(避免后续回溯改掉它)
        return
    for i in range(k, len(a)):
        a[k], a[i] = a[i], a[k]              # 把 a[i] 换到第 k 位
        permute(a, k + 1, result)            # 递归排列剩余部分
        a[k], a[i] = a[i], a[k]              # 回溯:换回来,还原现场
```

lab1.py(第二段:任务 3 计时与主程序)

```

# 任务 3:朴素递归斐波那契(用于计时)
def fib_naive(n):
    if n <= 1:
        return n
    return fib_naive(n - 1) + fib_naive(n - 2)

def main():
    global move_cnt
    print("==== 任务 1:汉诺塔(n = 3)====")
    hanoi(3, "A", "B", "C", True)
    print(f"移动步数: {move_cnt}(理论  $2^3 - 1 = 7$ )")
    move_cnt = 0 # 清零,再进入批量验证
    for n in range(1, 11): # 验证  $2^n - 1$ 
        hanoi(n, "A", "B", "C", False)
        expect = 2 ** n - 1
        print(f"n = {n}: 实际 {move_cnt},理论 {expect}",
              "一致" if move_cnt == expect else "不一致!")
        move_cnt = 0

    print("\n==== 任务 2:全排列(n = 3)====")
    result = []
    permute([1, 2, 3], 0, result)
    for p in result:
        print(" ".join(map(str, p)))
    print(f"排列总数: {len(result)}(理论  $3! = 6$ )")
    result = []
    permute([1, 2, 3, 4], 0, result) # 再验证  $4! = 24$ 
    print(f"n = 4 排列总数: {len(result)}(理论  $4! = 24$ )")

    print("\n==== 任务 3:朴素斐波那契计时 ====")
    for n in (28, 31, 34): # n 每次 +3,调用数约  $\times 4.2$ 
        t0 = time.perf_counter()
        f = fib_naive(n)
        ms = (time.perf_counter() - t0) * 1000
        print(f"fib({n}) = {f},耗时 {ms:.1f} ms")
    print("观察: n 每加 3,耗时约乘 4~5(调用数  $2 \times F(n+1)$  的比值约  $\phi^3 = 4.2$ )")

if __name__ == "__main__":
    main()

```

运行命令: python lab1.py 。示例输出(第一段,n = 3 的移动过程):

===== 任务 1:汉诺塔(n = 3)=====

A -> C
A -> B
C -> B
A -> C
B -> A
B -> C
A -> C

移动步数: 7(理论 $2^3 - 1 = 7$)

n = 1: 实际 1,理论 1 一致

n = 2: 实际 3,理论 3 一致

n = 3: 实际 7,理论 7 一致

n = 4: 实际 15,理论 15 一致

n = 5: 实际 31,理论 31 一致

n = 6: 实际 63,理论 63 一致

n = 7: 实际 127,理论 127 一致

n = 8: 实际 255,理论 255 一致

n = 9: 实际 511,理论 511 一致

n = 10: 实际 1023,理论 1023 一致

示例输出(第二段;耗时因机器而异,比值规律一致):

===== 任务 2:全排列(n = 3)=====

1 2 3
1 3 2
2 1 3
2 3 1
3 2 1
3 1 2

排列总数: 6(理论 $3! = 6$)

n = 4 排列总数: 24(理论 $4! = 24$)

===== 任务 3:朴素斐波那契计时 =====

fib(28) = 317811,耗时 23.0 ms

fib(31) = 1346269,耗时 103.8 ms

fib(34) = 5702887,耗时 433.7 ms

观察: n 每加 3,耗时约乘 4~5(调用数 $2 * F(n+1)$ 的比值约 $\phi^3 = 4.2$)

设计说明:① 三个任务共用"基准情形 + 不断推进"的递归骨架:汉诺塔的基准是 $n == 1$,全排列是"最后一位也固定",斐波那契是 $n \leq 1$ ——同一个思想的三次演练;② 汉诺塔的两次递归调用展示了"参数轮换"技巧,调用次数 $2^n - 1$ 正是第2章递推式 $T(n) = 2T(n-1) + O(1)$ 的实物原型;③ 全排列的"交换 → 递归 → 换回"是回溯法(第10章)的最小模板;④ 任务 3 用真实计时把 $O(2^n)$ 变成肉眼可见:本机 $n = 28/31/34$ 耗时 23/104/434 ms,每加 3 约乘 4~5(比值恰为 $\phi^3 \approx 4.2$);⑤ 打印与计数分离(print_move 开关)让"演示过程"与"批量验证"互不干扰;⑥ `time.perf_counter()` 是第2章所有实验的标准计时器,本实验提前练手。

下一章预告:第2章 算法分析——我们将为大 O 记号建立严格定义,学会对循环与递归做复杂度计算,并把最大子序列和问题从 $O(n^3)$ 一路优化到 $O(n)$ 。本章的递推式、对数与级数公式,正是第2章一切推导的数学地基。